Нижче: **(1) де брати реальні датасети + (2) готовий код-шаблон для Модуля 2** (збір → очистка → візуалізація → звіт).

1) Джерела датасетів (реальні)

Універсальні репозиторії (для навчальних задач)

- **UCI Machine Learning Repository** — класичні табличні датасети (класифікація/регресія). archive.ics.uci.edu+1
- **OpenML** — багато датасетів із метаданими, зручно для автоматизації. openml.org+1
- **Kaggle Datasets** — дуже широкий вибір "реальних" тем (e-commerce, медичні, спорт тощо). kaggle.com+1

Державні open-data портали

- **Україна: data.gov.ua** — тисячі публічних наборів (CSV/ZIP). data.gov.ua+1
- **США: Data.gov** — великий каталог урядових датасетів. Data.gov+1
- **ЄС: data.europa.eu** — європейський портал датасетів. data.europa.eu+1

API/стріми даних (круто для "збору" в модулі 2)

- **World Bank Indicators API** — ~16k показників (GDP, населення, інфляція тощо). Світовий банк+1
- **USGS Earthquakes GeoJSON feed** — землетруси у форматі GeoJSON. USGS+1
- **NOAA Climate Data Online (CDO)** — архіви погоди/клімату (часто потрібен токен). Наццентрінфоекосередовище+1

2) Код програми для Модуля 2 (шаблон)

Це **один скрипт** `module2.py`, який:

- **collect**: завантажує CSV або JSON(API) у `data/raw/`
- **clean**: чистить дані → `data/processed/clean.csv`
- **viz**: будує 4 графіки → `artifacts/figures/`
- **report**: робить короткий `report.md`

URL-и я даю **в коді** (так правильно для запуску).
Приклади джерел: **USGS** (землетруси), **World Bank** (індикатори), або ваш CSV.

```
# module2.py
# Python 3.10+
# pip install pandas numpy matplotlib requests

from __future__ import annotations
import argparse
import json
from pathlib import Path
from datetime import datetime

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import requests


RAW_DIR = Path("data/raw")
PROC_DIR = Path("data/processed")
FIG_DIR = Path("artifacts/figures")
REP_DIR = Path("artifacts/reports")


# -------------------- IO helpers --------------------
def ensure_dirs():
    for p in [RAW_DIR, PROC_DIR, FIG_DIR, REP_DIR]:
        p.mkdir(parents=True, exist_ok=True)

def save_json(obj: dict, path: Path):
    path.parent.mkdir(parents=True, exist_ok=True)
```

```
    path.write_text(json.dumps(obj, indent=2, ensure_ascii=False),
encoding="utf-8")

def load_any(path: Path) -> pd.DataFrame:
    if path.suffix.lower() == ".csv":
        return pd.read_csv(path)
    if path.suffix.lower() in [".json", ".geojson"]:
        return pd.read_json(path)
    raise ValueError(f"Unsupported format: {path.suffix}")


# -------------------- COLLECT --------------------
def collect_usgs_earthquakes(out_name="earthquakes_usgs.geojson"):
    # USGS all_day feed (GeoJSON)
    url =
"https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_day.geojson"
    r = requests.get(url, timeout=30)
    r.raise_for_status()
    out_path = RAW_DIR / out_name
    out_path.write_bytes(r.content)
    print(f"Saved: {out_path}")

def collect_worldbank_gdp(country="UKR", out_name="worldbank_gdp.json"):
    # World Bank API example: GDP (current US$) indicator NY.GDP.MKTP.CD
    url =
f"https://api.worldbank.org/v2/country/{country}/indicator/NY.GDP.MKTP.CD?format
=json"
    r = requests.get(url, timeout=30)
    r.raise_for_status()
    out_path = RAW_DIR / out_name
    out_path.write_text(r.text, encoding="utf-8")
    print(f"Saved: {out_path}")

def collect_from_url(url: str, out_name: str):
    r = requests.get(url, timeout=60)
    r.raise_for_status()
    out_path = RAW_DIR / out_name
    out_path.write_bytes(r.content)
    print(f"Saved: {out_path}")


# -------------------- CLEAN --------------------
def clean_dataframe(df: pd.DataFrame) -> tuple[pd.DataFrame, dict]:
    # Базова перевірка/очистка: дублі, пропуски, типи
    info = {}
    info["shape_before"] = list(df.shape)
    info["missing_before"] = int(df.isna().sum().sum())
    info["duplicates_before"] = int(df.duplicated().sum())

    df = df.drop_duplicates().copy()

    # Спроба: конвертувати колонки з датами
    for c in df.columns:
        if df[c].dtype == "object":
            # якщо схоже на дату
            try:
                parsed = pd.to_datetime(df[c], errors="raise", utc=True)
                # якщо конвертація успішна для більшості значень
                if parsed.notna().mean() > 0.8:
                    df[c] = parsed
            except Exception:
                pass

    # Числові колонки: примусово to_numeric там, де можливо
    for c in df.columns:
        if df[c].dtype == "object":
```

```
            conv = pd.to_numeric(df[c], errors="coerce")
            # якщо багато чисел — замінимо
            if conv.notna().mean() > 0.8:
                df[c] = conv

    # Заповнення пропусків: числа → медіана, категорії → мода
    for c in df.columns:
        if pd.api.types.is_numeric_dtype(df[c]):
            if df[c].isna().any():
                df[c] = df[c].fillna(df[c].median())
        else:
            if df[c].isna().any():
                mode = df[c].mode(dropna=True)
                df[c] = df[c].fillna(mode.iloc[0] if len(mode) else "UNKNOWN")

    info["shape_after"] = list(df.shape)
    info["missing_after"] = int(df.isna().sum().sum())
    info["duplicates_after"] = int(df.duplicated().sum())
    return df, info


# -------------------- VIZ --------------------
def pick_numeric_cols(df: pd.DataFrame, k=2) -> list[str]:
    cols = [c for c in df.columns if pd.api.types.is_numeric_dtype(df[c])]
    return cols[:k]

def pick_category_col(df: pd.DataFrame) -> str | None:
    cats = [c for c in df.columns if df[c].dtype == "object"]
    return cats[0] if cats else None

def visualize(df: pd.DataFrame, prefix="plot"):
    # Мінімум 4 графіки:
    # 1) гістограма, 2) scatter, 3) bar групування, 4) кореляція (якщо >=2
numeric)
    num = pick_numeric_cols(df, k=2)
    cat = pick_category_col(df)

    if not num:
        raise ValueError("No numeric columns found for plots.")

    # 1) Histogram
    plt.figure()
    df[num[0]].plot(kind="hist", bins=30)
    plt.title(f"Histogram: {num[0]}")
    p1 = FIG_DIR / f"{prefix}_hist.png"
    plt.savefig(p1, bbox_inches="tight")
    plt.close()

    # 2) Scatter (якщо є друга числова)
    if len(num) >= 2:
        plt.figure()
        plt.scatter(df[num[0]], df[num[1]])
        plt.title(f"Scatter: {num[0]} vs {num[1]}")
        plt.xlabel(num[0]); plt.ylabel(num[1])
        p2 = FIG_DIR / f"{prefix}_scatter.png"
        plt.savefig(p2, bbox_inches="tight")
        plt.close()
    else:
        p2 = None

    # 3) Bar (топ-10 категорій за середнім num[0])
    p3 = None
    if cat is not None:
        g = df.groupby(cat,
dropna=False)[num[0]].mean().sort_values(ascending=False).head(10)
        plt.figure()
```

```
        g.plot(kind="bar")
        plt.title(f"Top-10 {cat} by mean {num[0]}")
        plt.ylabel(f"mean({num[0]})")
        p3 = FIG_DIR / f"{prefix}_bar.png"
        plt.savefig(p3, bbox_inches="tight")
        plt.close()

    # 4) Correlation heatmap (простий через imshow)
    p4 = None
    if len(num) >= 2:
        corr = df[[c for c in df.columns if
pd.api.types.is_numeric_dtype(df[c])]].corr(numeric_only=True)
        plt.figure()
        plt.imshow(corr.values)
        plt.title("Correlation (numeric)")
        plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
        plt.yticks(range(len(corr.columns)), corr.columns)
        plt.colorbar()
        p4 = FIG_DIR / f"{prefix}_corr.png"
        plt.savefig(p4, bbox_inches="tight")
        plt.close()

    return {"hist": str(p1), "scatter": str(p2) if p2 else None, "bar": str(p3)
if p3 else None, "corr": str(p4) if p4 else None}


# -------------------- REPORT --------------------
def make_report(raw_path: Path, clean_path: Path, clean_info: dict, fig_paths:
dict):
    ts = datetime.now().strftime("%Y-%m-%d %H:%M")
    lines = []
    lines.append(f"# Module 2 Report")
    lines.append(f"- Time: {ts}")
    lines.append(f"- Raw file: {raw_path}")
    lines.append(f"- Clean file: {clean_path}")
    lines.append("")
    lines.append("## Cleaning summary")
    lines.append("```json")
    lines.append(json.dumps(clean_info, indent=2, ensure_ascii=False))
    lines.append("```")
    lines.append("")
    lines.append("## Figures")
    for k, v in fig_paths.items():
        if v:
            lines.append(f"- **{k}**: {v}")
    lines.append("")
    lines.append("## Conclusions (fill by student)")
    lines.append("1) ...")
    lines.append("2) ...")
    lines.append("3) ...")

    out = REP_DIR / "report.md"
    out.write_text("\n".join(lines), encoding="utf-8")
    print(f"Saved: {out}")


# -------------------- CLI --------------------
def main():
    ensure_dirs()

    ap = argparse.ArgumentParser()
    sub = ap.add_subparsers(dest="cmd", required=True)

    c1 = sub.add_parser("collect_usgs")
    c1.add_argument("--out", default="earthquakes_usgs.geojson")
```

```
    c2 = sub.add_parser("collect_worldbank")
    c2.add_argument("--country", default="UKR")
    c2.add_argument("--out", default="worldbank_gdp.json")

    c3 = sub.add_parser("collect_url")
    c3.add_argument("--url", required=True)
    c3.add_argument("--out", required=True)

    cl = sub.add_parser("clean")
    cl.add_argument("--infile", required=True)
    cl.add_argument("--outfile", default="data/processed/clean.csv")

    vz = sub.add_parser("viz")
    vz.add_argument("--infile", required=True)
    vz.add_argument("--prefix", default="module2")

    rp = sub.add_parser("report")
    rp.add_argument("--raw", required=True)
    rp.add_argument("--clean", required=True)
    rp.add_argument("--cleaninfo", required=True)  # path to json
    rp.add_argument("--prefix", default="module2")

    args = ap.parse_args()

    if args.cmd == "collect_usgs":
        collect_usgs_earthquakes(args.out)

    elif args.cmd == "collect_worldbank":
        collect_worldbank_gdp(args.country, args.out)

    elif args.cmd == "collect_url":
        collect_from_url(args.url, args.out)

    elif args.cmd == "clean":
        raw_path = Path(args.infile)
        df = load_any(raw_path)

        # Спец-обробка USGS GeoJSON (якщо це earthquakes_usgs.geojson)
        if raw_path.suffix.lower() == ".geojson" and "features" in df.columns:
            # df буде "плоским" некоректно; краще розпарсити вручну:
            geo = json.loads(raw_path.read_text(encoding="utf-8"))
            rows = []
            for f in geo.get("features", []):
                p = f.get("properties", {})
                rows.append({
                    "time": p.get("time"),
                    "place": p.get("place"),
                    "mag": p.get("mag"),
                    "tsunami": p.get("tsunami"),
                    "felt": p.get("felt"),
                    "type": p.get("type"),
                })
            df = pd.DataFrame(rows)
            # time у мілісекундах
            if "time" in df.columns:
                df["time"] = pd.to_datetime(df["time"], unit="ms", utc=True,
errors="coerce")

        # Спец-обробка WorldBank (JSON масив [meta, data])
        if raw_path.suffix.lower() == ".json":
            try:
                wb = json.loads(raw_path.read_text(encoding="utf-8"))
                if isinstance(wb, list) and len(wb) == 2 and isinstance(wb[1],
list):
                    df = pd.json_normalize(wb[1])
            except Exception:
```

```
                pass

        clean_df, info = clean_dataframe(df)
        out_path = Path(args.outfile)
        out_path.parent.mkdir(parents=True, exist_ok=True)
        clean_df.to_csv(out_path, index=False)

        info_path = REP_DIR / "clean_info.json"
        save_json(info, info_path)

        print(f"Saved: {out_path}")
        print(f"Saved: {info_path}")

    elif args.cmd == "viz":
        df = pd.read_csv(args.infile)
        figs = visualize(df, prefix=args.prefix)
        save_json(figs, REP_DIR / "fig_paths.json")
        print("Saved figure paths to artifacts/reports/fig_paths.json")

    elif args.cmd == "report":
        clean_info = json.loads(Path(args.cleaninfo).read_text(encoding="utf-
8"))
        fig_paths_path = REP_DIR / "fig_paths.json"
        fig_paths = json.loads(fig_paths_path.read_text(encoding="utf-8")) if
fig_paths_path.exists() else {}
        make_report(Path(args.raw), Path(args.clean), clean_info, fig_paths)

if __name__ == "__main__":
    main()
```

Як запускати (приклади)

```
# 1) Збір (USGS землетруси)
python module2.py collect_usgs --out earthquakes_usgs.geojson

# 2) Очистка → processed/clean.csv + clean_info.json
python module2.py clean --infile data/raw/earthquakes_usgs.geojson --outfile
data/processed/clean.csv

# 3) Візуалізація (4 графіки)
python module2.py viz --infile data/processed/clean.csv --prefix earthquakes

# 4) Звіт
python module2.py report --raw data/raw/earthquakes_usgs.geojson --clean
data/processed/clean.csv --cleaninfo artifacts/reports/clean_info.json
```